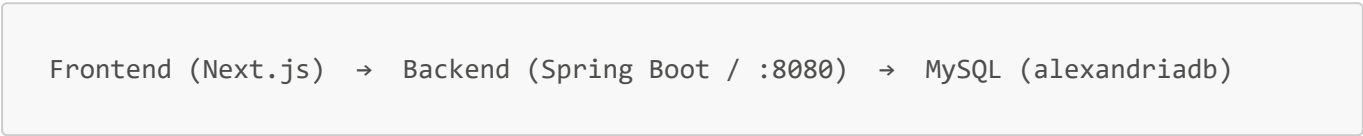


Fluxo de Dados — Página Explorar

Visão Geral

Os livros exibidos na página Explorar vêm do backend Spring Boot via API REST, que por sua vez lê do banco de dados MySQL.



1. Banco de Dados

Banco: MySQL

Config: alexandria-backend/src/main/resources/application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/alexandriadb
spring.datasource.username=root
spring.datasource.password=root
```

2. Backend — Endpoints

Método	Endpoint	Descrição
GET	/books?page=0&size=10	Lista paginada de livros
GET	/books/search?query=...&page=0&size=10	Busca por título/autor/assunto

Resposta segue o formato paginado do Spring Data:

```
{
  "content": [ ... ],
  "totalPages": 5,
  "totalElements": 48,
  "number": 0,
  "size": 10
}
```

3. Frontend — Camada de API

Arquivo: alexandria-frontend/src/lib/api.ts

```
// Listagem
getBooks(page = 0, size = 10): Promise<BooksPage>
// GET $NEXT_PUBLIC_API_URL/books?page=0&size=10

// Busca
searchBooks(query, page = 0, size = 10): Promise<BooksPage>
// GET $NEXT_PUBLIC_API_URL/books/search?query=...
```

A URL base é configurada pela variável de ambiente `NEXT_PUBLIC_API_URL` (ex.: `http://localhost:8080`).

Tipo de retorno:

```
interface BookApiResponse {
  id: number;
  title: string;
  author: string;
  coverUrl: string | null;
  downloadUrl: string | null;
  languages: string | null;
  subjects: string | null;
  source: string;
  gutendexId: number | null; // indica origem no Project Gutenberg
  downloadCount: number | null;
  publisherId: number | null;
}
```

4. Frontend — Página Explorar

Arquivo: `alexandria-frontend/src/app/(main)/explorar/page.tsx`

```
// Carregamento inicial (linha 14-19)
useEffect(() => {
  getBooks(0, 10)
    .then((page) => setBooks(page.content))
    .catch(console.error)
    .finally(() => setLoading(false));
}, []);
```

- Ao montar o componente, busca os 10 primeiros livros.
- Durante a carga, exibe esqueletos animados (`animate-pulse`).
- Submissão do formulário de busca chama `searchBooks`; limpar a busca volta a chamar `getBooks`.
- Os livros são renderizados via `<BookCard>` (linha 237-239).

5. Origem dos Dados no Banco

O campo `gutendexId` indica que o backend importa/sincroniza livros da API pública **Gutendex** (Project Gutenberg). Essa importação é feita pelo backend Java — o frontend nunca chama o Gutendex diretamente.

6. Implementação do Leitor de EPUB

Resumo

Foi implementado um fluxo completo de leitura de livros em formato EPUB, integrando a página de detalhes do livro com um leitor embutido no próprio app. O leitor carrega o arquivo EPUB remotamente (via Project Gutenberg) através de uma rota de proxy no próprio servidor Next.js, evitando problemas de CORS.

Bibliotecas adicionadas

Biblioteca	Versão	Finalidade
<code>react-reader</code>	<code>^2.0.15</code>	Renderização de EPUB no navegador via <code>epub.js</code> . Exibe o conteúdo paginado, mantém posição de leitura e suporta navegação por capítulos.

Arquivos novos

`alexandria-frontend/src/app/api/epub/route.ts`

Rota de API (Route Handler do Next.js) que funciona como **proxy reverso** para arquivos EPUB externos.

Por que existe: o navegador não pode buscar arquivos de outros domínios diretamente (bloqueio CORS). A rota roda no servidor e repassa o binário do EPUB para o cliente.

Fluxo:

1. Recebe `?url=<endereço do EPUB>` via query string
2. Faz `fetch` server-side para a URL informada (ex.: Gutenberg)
3. Retorna o `ArrayBuffer` com `Content-Type: application/epub+zip`

```
GET /api/epub?url=<url_do_epub>
→ Next.js server faz fetch → Gutenberg (ou outra fonte)
→ retorna binário EPUB para o cliente
```

`alexandria-frontend/src/app/(main)/leitor/[id]/page.tsx`

Página de leitura dinâmica, acessível via `/leitor/:id`.

Fluxo:

1. Busca os dados do livro via `getBookById(id)` (`/books/:id` no backend)
2. Usa `localStorage` para salvar e restaurar a posição de leitura por livro (`epub-location-<id>`)
3. Faz `fetch` para `/api/epub?url=<downloadUrl>` e armazena o `ArrayBuffer` em estado
4. Renderiza o `<ReactReader>` com o binário do EPUB

Estados de UI:

- Carregando leitor... — enquanto os dados do livro são buscados
 - Carregando livro... — dados do livro prontos, EPUB ainda não carregado
 - Arquivo de leitura indisponível — livro sem `downloadUrl`
 - Leitor completo — EPUB carregado com sucesso
-

Arquivos modificados

`alexandria-frontend/src/app/(main)/explorar/[id]/page.tsx`

Página de detalhes do livro. GANHOU o botão **"Ler agora"** que navega para `/leitor/:id`. O botão só é exibido quando `book.downloadUrl` está preenchido; caso contrário, exibe "Leitura indisponível" desabilitado.

`alexandria-frontend/src/lib/api.ts`

Adicionados:

- Interface `BookApiResponse` — tipagem completa do objeto livro retornado pelo backend
- Interface `BooksPage` — wrapper paginado do Spring Data
- Funções `getBooks`, `searchBooks`, `getBookById` — chamadas REST ao backend
- Funções `getAuthHeaders` e `apiFetch` — helper com token JWT nos headers

`alexandria-frontend/src/components/BookCard.tsx`

Componente de card de livro refatorado para receber `BookApiResponse` diretamente da API. Inclui link para `/explorar/:id` e botão "Adicionar à Coleção" com callback opcional `onAdd`.

`alexandria-frontend/package.json`

Adicionada dependência `react-reader: ^2.0.15`.

Fluxo completo — Explorar → Detalhe → Leitor

```
/explorar
→ lista livros via GET /books (backend :8080)
→ <BookCard> com link para /explorar/:id

/explorar/:id
→ GET /books/:id (backend)
→ exibe capa, título, autor, assuntos, idioma
→ botão "Ler agora" → /leitor/:id (só se downloadUrl existir)

/leitor/:id
→ GET /books/:id (backend) — busca downloadUrl
→ GET /api/epub?url=<downloadUrl> (proxy Next.js)
  → fetch server-side para Gutenberg
  → retorna binário EPUB
```

- `<ReactReader>` renderiza o EPUB
- posição salva/restaurada via `localStorage`